SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: ITA 06 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS:15 **Total Marks:25**

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

I R SARVESH / 192521143 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

R SARVESH

Annexure A

Short Answer Test

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence. (5 Marks)

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance. (5 Marks)

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed. (5 Marks)

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability. (5 Marks)

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization. (5 Marks)

6. A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy. (5 Marks)

7. A reinforcement learning agent fails to learn an optimal policy in a simulated environment. Investigate causes such as poor reward design, insufficient exploration, or incorrect state representation. Explain debugging methods and recommend improvements like reward shaping, exploration strategies, and tuning hyperparameters. (5 Marks)

8. A clustering algorithm like K-means produces inconsistent cluster assignments for similar datasets. Analyze possible reasons such as poor initialization, incorrect number of clusters, or sensitivity to outliers. Propose debugging approaches and optimization strategies like K-means++, normalization, and silhouette analysis. (5 Marks)

9. A support vector machine (SVM) model underperforms on a nonlinear dataset. Identify issues such as incorrect kernel choice, improper parameter tuning, or scaling problems. Suggest debugging techniques and optimization methods including kernel selection, hyperparameter tuning, and feature scaling. (5 Marks)

10. A natural language processing (NLP) model using recurrent neural networks (RNNs) suffers from poor long-term dependency learning. Examine causes such as vanishing gradients and limited memory capacity. Describe debugging steps and propose solutions like LSTM/GRU units, attention mechanisms, and improved training strategies. (5 Marks)

11. A support vector machine (SVM) model trained for classification is producing very low accuracy on both training and test datasets. Analyze possible causes such as improper kernel selection, incorrect hyperparameter tuning (C and gamma), or poor feature scaling. Describe systematic debugging steps to identify the issue and suggest optimization techniques like kernel tuning, normalization, and grid search to improve model performance. (5 Marks)

12. A K-Means clustering algorithm is applied to a dataset, but the clusters formed are highly imbalanced and do not reflect the actual data distribution. Investigate possible issues such as incorrect choice of K, poor initialization of centroids, or presence of outliers. Explain debugging approaches and suggest improvements like the Elbow method, K-Means++, and outlier removal techniques. (5 Marks)

13. A recurrent neural network (RNN) designed for time-series prediction shows inconsistent outputs and fails to capture long-term dependencies. Analyze possible causes such as vanishing gradients, insufficient sequence length, or improper architecture design. Describe debugging steps and propose optimization techniques such as using LSTM/GRU units, gradient clipping, and sequence normalization. (5 Marks)

14. A Naïve Bayes classifier used for text classification performs poorly when applied to real-world data containing noisy and irrelevant features. Examine possible issues such as violation of feature independence assumption, improper preprocessing, or imbalanced dataset. Explain debugging strategies and suggest improvements like feature selection, smoothing techniques, and data balancing methods. (5 Marks)

15. A reinforcement learning agent trained to solve a navigation problem fails to learn an optimal policy and shows erratic behavior. Analyze possible causes such as sparse rewards, improper exploration-exploitation balance, or incorrect reward function design. Describe debugging approaches and recommend optimization strategies like reward shaping, epsilon decay tuning, and policy optimization methods to improve learning performance. (5 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**20**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**30**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**10**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q1. Perceptron Fails to Converge

A perceptron is a supervised learning algorithm used for binary classification. If the dataset is linearly separable but the perceptron does not converge, there may be errors in its configuration or implementation.

Possible causes:

1. Improper learning rate: A very large learning rate may cause unstable updates.
2. Incorrect weight initialization: Poor initial weights may slow down learning.
3. Bias handling errors: An incorrect bias value or failure to update the bias can affect classification.
4. Incorrect update rule: Wrong error calculation or incorrect weight updates can prevent convergence.
5. Data issues: Incorrect labels or improperly processed data may prevent the expected result.

Debugging and optimization:

- Verify the training data and class labels.
- Check the weight and bias update calculations.
- Normalize input features when their scales differ significantly.
- Use a suitable learning rate and monitor classification errors.
- Confirm that the data is actually linearly separable.

Conclusion: Correct implementation, proper data preparation, and suitable parameter settings help the perceptron converge efficiently.

Q2. Slow Convergence in a Multilayer Perceptron

A multilayer perceptron (MLP) is a neural network trained using backpropagation to reduce prediction error. Slow convergence occurs when the network learns very slowly or struggles to reach a good solution.

Possible causes:

1. Vanishing gradients: Gradients become very small, preventing early layers from learning effectively.
2. Poor activation functions: Sigmoid activation can cause vanishing gradients in deep networks.
3. Poor weight initialization: Improper weights may slow down training.

4. Incorrect learning rate: A very low rate slows learning, while a very high rate causes unstable updates.
5. Unnormalized data: Different feature scales can make training difficult.

Debugging and optimization:

- Monitor training and validation loss.
- Check gradients for vanishing or exploding values.
- Normalize input features.
- Use ReLU activation and suitable weight initialization.
- Apply Adam or another adaptive optimizer and tune the learning rate.

Conclusion: Proper initialization, normalization, activation functions, and optimizers improve convergence speed and model performance.

Q5. Decision Tree Overfitting

Overfitting occurs when a decision tree learns the training data too closely but performs poorly on unseen data.

Possible causes:

1. Excessive tree depth: A very deep tree learns unnecessary details.
2. Noisy data: Incorrect or irrelevant records mislead the model.
3. Too many features: Unnecessary features may create poor splits.
4. Small leaf sizes: The model may create branches based on very few samples.

Debugging and optimization:

- Compare training accuracy with validation accuracy.
- Use cross-validation to assess generalization.
- Apply pruning to remove unnecessary branches.
- Limit the maximum tree depth.
- Perform feature selection and remove noisy data.

Conclusion: Pruning, cross-validation, and feature selection reduce overfitting and improve the decision tree's ability to predict unseen data.

Q8. Inconsistent K-Means Clustering

K-Means is an unsupervised learning algorithm that divides data into K clusters based on similarity. Inconsistent clusters may occur because of initialization or data-related problems.

Possible causes:

1. Poor initialization: Random starting centroids can produce different clusters.
2. Incorrect K: Choosing too many or too few clusters gives poor results.
3. Outliers: Extreme values can shift cluster centroids.
4. Different feature scales: Features with large values may dominate distance calculations.

Debugging and optimization:

- Normalize or standardize the features.
- Use K-Means++ to select better initial centroids.
- Apply the Elbow method to estimate a suitable K.
- Use silhouette analysis to evaluate cluster quality.
- Identify and handle outliers before clustering.

Conclusion: Proper initialization, feature scaling, and selecting a suitable number of clusters improve the consistency and quality of K-Means clustering.

Q9. SVM Underperforms on Nonlinear Data

A Support Vector Machine (SVM) is a supervised learning algorithm used for classification. It may perform poorly on nonlinear data if the kernel or model parameters are unsuitable.

Possible causes:

1. Incorrect kernel: A linear kernel may not capture complex nonlinear patterns.
2. Improper C value: A very high or low C can lead to overfitting or underfitting.
3. Incorrect gamma: In an RBF kernel, gamma controls the influence of individual training points.
4. Unscaled features: Features with different scales can affect the decision boundary.
5. Insufficient training data: The model may not learn the underlying patterns effectively.

Debugging and optimization:

- Standardize or normalize the features.
- Try an RBF kernel for nonlinear data.

- Tune C and gamma using grid search and cross-validation.
- Compare training and validation performance.
- Check data quality and class balance.

Conclusion: Appropriate kernel selection, feature scaling, and hyperparameter tuning help improve SVM classification accuracy.